

Document Summary

This note explores the fundamentals of Recurrent Neural Networks (RNNs), highlighting their advantages over fully connected neural networks for sequential data processing. It delves into key challenges like vanishing gradients and long-term dependencies, and advanced solutions such as LSTMs and encoder-decoder architectures. Practical examples, including a case correction demo, illustrate RNN training and limitations.

Recurrent Neural Networks: Fundamentals, Challenges, and Advanced Architectures (LSTMs, Encoder-Decoder)

Limitations of Fully Connected Neural Networks (FCNNs)

Fully Connected Neural Networks (FCNNs)

FCNNs are cascades of matrix multiplications combined with non-linear activation functions, such as $f(x) = \sigma(W_{out} \text{ReLU}(W_{hid2} \text{ReLU}(W_{hid1}x)))$, where σ is a non-linear activation function, ReLU is the rectified linear unit, and W terms represent weight matrices.

However, these models come with several architectural constraints that limit their applicability, especially for sequential or variable-length data. Let's examine these limitations step by step.

1. Fixed Input Size Requirement:

The input to an FCNN must be fixed in size, which is determined by the number of columns in the input weight matrix W_{in} .

- This works well for structured data, such as tabular datasets, where there is a predefined number of input features or attributes.
- In such cases, missing values can be managed through techniques like imputation or padding.
- For unstructured data, however, this becomes a significant problem because inputs often vary in structure:

- Images can have different resolutions or aspect ratios.
- Audio signals may differ in sampling frequencies and durations.
- Text sequences come in varying lengths.
- To fit them into an FCNN, flattening is commonly required, such as reshaping a 2D image from dimensions $d_1 \times d_2$ into a 1D vector of size $d_1 \cdot d_2$. This process loses spatial or temporal relationships, making the model less effective.

2. Dedicated Weights for Each Input Position:

In FCNNs, each input element is processed using its own unique set of weights.

- This means that if a similar pattern appears in different parts of the input, the model must learn to recognize and handle that pattern independently in each location.
- Such redundancy becomes computationally expensive, particularly when patterns repeat across multiple positions.
- **Example:** Consider image classification. A dog appearing on the right side of an image requires the model to learn "dog on the right," while one on the left requires learning "dog on the left" separately. This inefficiency demands more training data and increases the risk of overfitting.

3. Fixed Output Size Requirement:

Similarly, the output of an FCNN is constrained to a fixed size, defined by the number of rows in the output weight matrix W_{out} .

- This poses challenges when the desired output varies in size or structure.
- **Example:** In object detection tasks, the model might need to segment an image into variable regions, such as labeling pixels as "Dog" or "Not dog." The number of such segments isn't fixed, so an FCNN struggles to produce outputs of arbitrary length.

Key Insight

*These limitations highlight why FCNNs are ill-suited for sequential data, where order and variability are key. This naturally leads to the need for **Recurrent Neural Networks (Neural Networks)**, which are designed to handle such dynamics.*

Fundamentals of Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs)

*RNNs are a specialized class of **Neural Networks** tailored for processing sequential data. Unlike FCNNs, RNNs can accept sequences of varying lengths as input and generate sequences of*

varying lengths as output, making them ideal for tasks like time series forecasting, natural language processing, and speech recognition.

In essence, an RNN processes input elements one at a time—for example, a word in a sentence, a character in a text string, or a daily measurement in a time series—while maintaining an internal representation of the information encountered so far. This internal state allows the network to capture dependencies across the sequence.

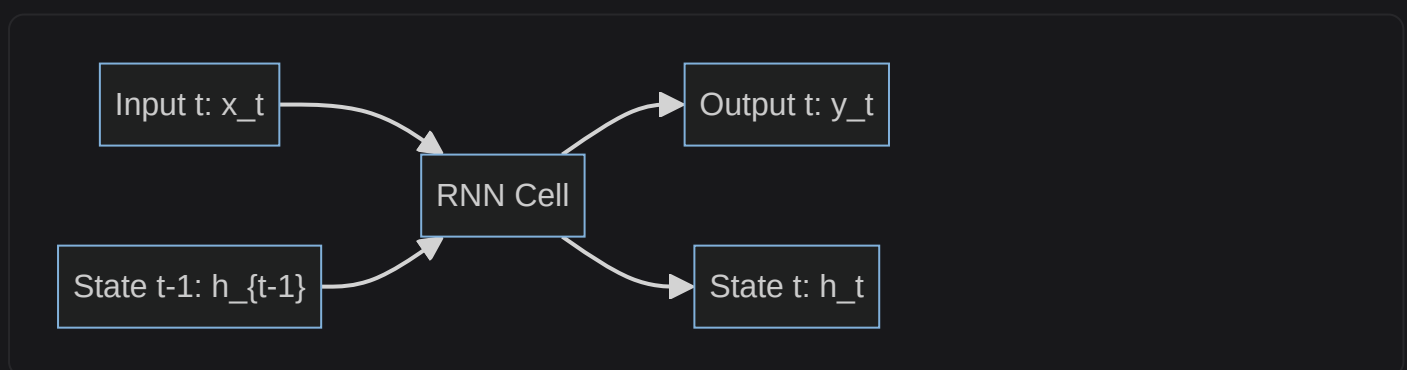
RNN Architecture

The core architecture of an RNN involves two inputs and two outputs at each time step, enabling it to build context progressively. To clarify:

- **First Input:** The current element of the input sequence at time step t , denoted as x_t .
- **Second Input:** The hidden state from the previous time step, h_{t-1} , which carries forward the summary of prior information.
- **First Output:** The prediction or output for the current time step, y_t .
- **Second Output:** The updated hidden state at time step t , h_t , which will serve as the second input for the next step.

This setup creates a loop, where the output of one step feeds into the next, allowing the RNN to process sequences dynamically.

For visual clarity, here's a Mermaid diagram representing the RNN architecture at a single time step:



RNN Architecture, Unfolded

When handling an entire sequence, the RNN is "unfolded" in time, applying the same cell repeatedly across each element. This unfolding reveals the recurrent nature, where weights are shared across steps.

Practical Example: Sequence Unfolding

Suppose we train an RNN to correct the casing of a lowercase input string, producing the properly cased version. Consider the input sequence: "you are very kind, mr. holmes, but i cannot do that". The desired output is: "You are very kind, Mr. Holmes, but I cannot do that".

Each character in the sequence is encoded as a 256-dimensional one-hot vector (one dimension per possible ASCII character). The process unfolds as follows:

- $s_0 = \text{" "}$ (opening quote)
- $s_1 = y$
- $s_2 = o$
- $s_3 = u$
- ... (continuing through the sequence)

At each step t :

- **At $t = 0$:** Start with an initial hidden state (all zeros) combined with $s_0 = \text{" "}$ to produce h_0 . This state might encode that a quote signals the start of dialogue, prompting capitalization of the next letter. The output is "\$".
- **At $t = 1$:** Feed h_0 and $s_1 = y$ into the RNN to get h_1 and output "Y" (capitalized, as it's after the quote).
- **At $t = 2$:** Use h_1 and $s_2 = o$ to produce h_2 and output "o" (lowercase, as expected mid-word).

This step-by-step unfolding allows the RNN to build context, such as remembering recent punctuation to decide on capitalization.

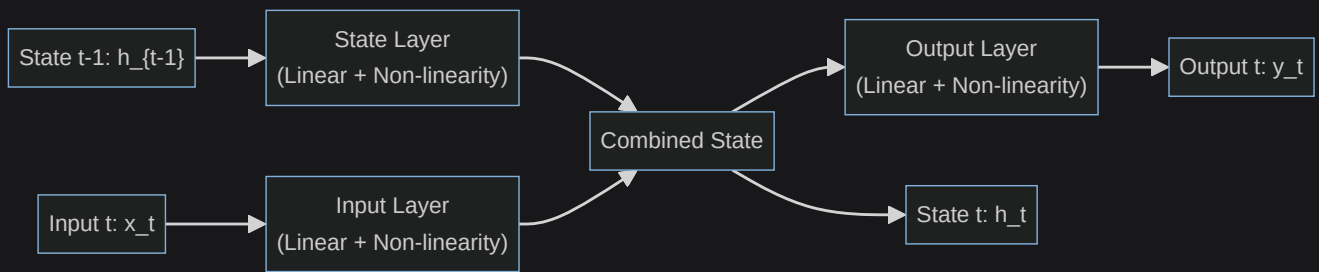
Inside an RNN

Various implementations of RNNs exist, but a basic one consists of three main layers, each applying linear transformations followed by non-linearities:

- **Input Layer:** Takes the current input x_t and projects it into a vector space matching the hidden state's dimensionality.
- **State Layer:** Processes the previous hidden state h_{t-1} to update the context.
- **Output Layer:** Combines the processed input and state to generate y_t , often via a final linear projection and softmax for probabilities.

Non-linearities, such as tanh or ReLU, are applied at each stage to introduce complexity.

For a clearer view of the internal flow:

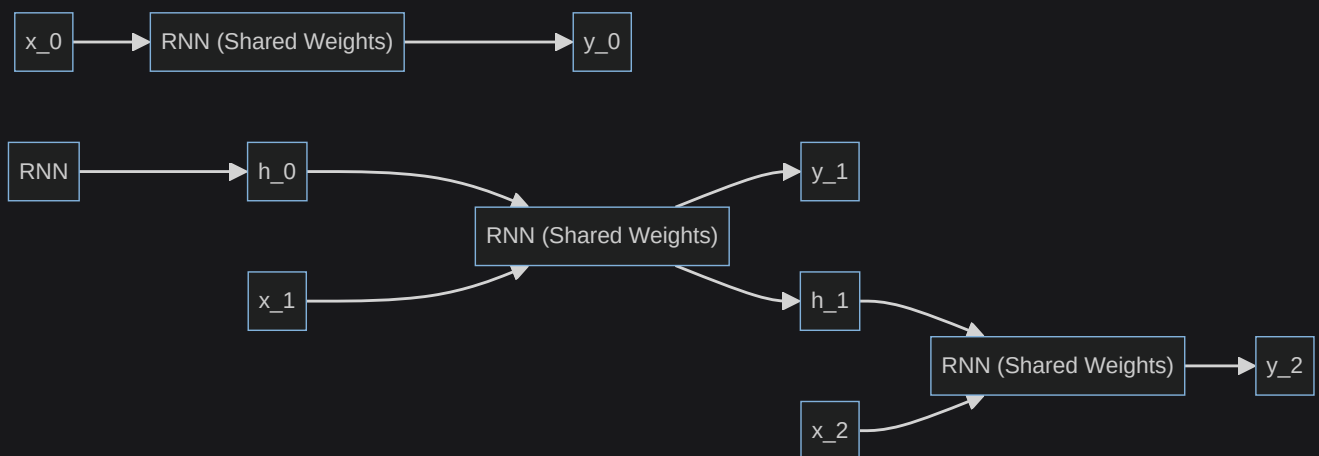


Weight Sharing

A key advantage of RNNs is weight sharing: The identical set of weights (for input, state, and output layers) is reused at every time step across the sequence.

- The hidden state h_t carries contextual information from prior steps, ensuring consistency.
- If a similar context arises at different points in the sequence, the RNN responds uniformly, without needing position-specific learning.
- **Example:** In text processing, the rule to capitalize the first letter after a period applies regardless of the sentence's position in a document. The shared weights enforce this consistency efficiently.

This is depicted as a chain of identical RNN cells:



Backpropagation Through Time (BPTT)

Training RNNs with BPTT

Training an RNN involves treating the unfolded network as a deep feedforward network, where gradients are computed across time steps using backpropagation through time (BPTT).

During training:

1. The RNN is unfolded over the full sequence length.
2. The total loss $\mathcal{L}$ is the sum of per-step losses.
3. Gradients are propagated backward, accumulating contributions from all time steps.

BPTT Example

For a sequence of length 4:

- *Inputs:* x_0, x_1, x_2, x_3
- *Outputs:* y_0, y_1, y_2, y_3
- *States:* Initial state $\rightarrow h_0 \rightarrow h_1 \rightarrow h_2 \rightarrow h_3$
- *Per-step losses:* $\mathcal{L}_0, \mathcal{L}_1, \mathcal{L}_2, \mathcal{L}_3$

The gradient for a weight W (e.g., input-to-hidden weights) includes terms like $\nabla_W(t_i, \mathcal{L}_j)$ for each pair of time steps i and loss j . The total gradient is the sum: $\nabla_W = \sum_{i,j} \nabla_W(t_i, \mathcal{L}_j)$. A single gradient descent update is then applied to all shared weights.

This accumulation ensures that even though the same weights are used multiple times, their updates reflect the entire sequence's influence. For a mathematical illustration, consider a simplified chain rule for the gradient with respect to an early weight:

Gradient Chain Rule

$$\frac{\partial \mathcal{L}_3}{\partial W_0} = \frac{\partial \mathcal{L}_3}{\partial y_3} \cdot \frac{\partial y_3}{\partial h_3} \cdot \frac{\partial h_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial h_0} \cdot \frac{\partial h_0}{\partial W_0}$$

Here, the product of terms $\frac{\partial h_t}{\partial h_{t-1}}$ can lead to vanishing or exploding values over long chains, as discussed later.

RNN Demo: Case Correction

RNN Demo: Case Correction

To demonstrate RNNs in action, consider a practical example using text from The Adventures of Sherlock Holmes by Arthur Conan Doyle (public domain). The dataset consists of randomly cropped sequences of 10-100 ASCII characters, such as "My cabby drove fast. I don't think I ever drove faster, but the others".

- **Input:** Lowercase versions of the sequences, fed one ASCII character at a time.
- **Target:** The original sequences with correct casing and punctuation.
- **Model:** A simple RNN with a 32-dimensional hidden state (initialized to all zeros), trained using the Adam optimizer with a learning rate of 0.001.
- **Encoding:** Each character is represented as a 256-dimensional one-hot vector.
- **Loss Function:** Cross-entropy loss between the predicted probability distribution over characters and the true next character.
- **Training Setup:** 10 epochs with a batch size of 32.

The training loss decreases over epochs, and we evaluate progress by testing on a fixed sentence after each epoch:

Input test sentence: "upon his pale face. "it may be so, or it may not, mr. holmes ," said he, "but if you are so very sharp"

Target: "upon his pale face. "It may be so, or it may not, Mr. Holmes," said he, "but if you are so very sharp"

For better readability, the epoch results are summarized in the following table:

Epoch	Loss	Predicted Output Excerpt (Key Improvements)
01	3.1681	XX (predicts null bytes/padding frequently)
02	2.3334	eXX; (starts predicting common letter 'e')
03	1.7436	tst X s t e e e e t e e t e s e e e s e e e e e e e e e XXXe e st s ss e t ee se eee e e s
04	0.9519	upon his pale face. ci t may be so s or it may not s m r . h ol r es st said he s c bul punctuation are inconsistent)
05	0.3579	upon his pale face. " i t may be so, or it may not, m r . h olmes ," said he, "but if y
06	0.1599	upon his pale face. " i t may be so, or it may not, m r . h olmes ," said he, "but if y
07	0.1009	upon his pale face. "It may be so, or it may not, m r . h olmes ," said he, "but if y
08	0.0752	upon his pale face. "It may be so, or it may not, m r . h olmes ," said he, "but if y
09	0.0599	upon his pale face. "It may be so, or it may not, m r . Holmes," said he, "but if yo
10	0.0515	upon his pale face. "It may be so, or it may not, m r . Holmes," said he, "but if yo

- **Early Epochs:** The model defaults to predicting frequent or padded elements like null bytes or 'e', as these minimize initial loss.
- **Mid-Epochs:** A breakthrough occurs where the RNN learns to forward most lowercase letters unchanged and begins handling basic punctuation.
- **Later Epochs:** Capitalization rules emerge, such as uppercasing the letter after a period (even if followed by a quote). For instance, 'Holmes' is capitalized because it follows a period, not due to name recognition—the model relies on local context.
- **Limitation Highlighted:** The model cannot capitalize "Mr." perfectly because it processes sequentially and doesn't "foresee" that "m" starts a title; it depends on patterns learned from training data.

This demo shows how RNNs progressively learn sequential patterns, though they have inherent challenges. See [Machine Learning](#) for broader context.

Challenges of RNNs

While RNNs excel at capturing short-term dependencies, they encounter significant hurdles, particularly with longer sequences. These issues stem from their recurrent structure and training dynamics.

- **Vanishing & Exploding Gradients:**

Gradient Computation in BPTT

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial y_T} \cdot \frac{\partial y_T}{\partial h_T} \cdots \frac{\partial h_{t+1}}{\partial h_t} \cdot \frac{\partial h_t}{\partial \theta}$$

for a parameter θ at early time t and final loss $\mathcal{L}$ at step T . The network is unrolled N times sequence length N , so gradients involve products of N terms $\frac{\partial h_{t+1}}{\partial h_t}$.

- If the absolute value of these derivatives is mostly less than 1 (common with sigmoid activations, where derivatives are ≤ 0.25), gradients vanish exponentially—e.g., $0.9^{100} \approx 0.005$ —making early weights nearly untrainable.
- **Numerical Example:** For $N = 50$ and derivative 0.95, the product is $0.95^{50} \approx 0.077$, already small; at $N = 100$, it's ≈ 0.006 .
- Conversely, if derivatives exceed 1 on average (e.g., $1.05^{100} \approx 131$), gradients explode, causing unstable training and NaN values.
- **Consequence:** Weight updates become negligible (vanishing) or erratic (exploding), hindering learning of long-range dependencies.

- **Long-Term Dependency Issues:**

Ideally, the hidden state $h_t = f(x_t, h_{t-1})$ should encapsulate the entire history up to t .

However, in practice:

- For long sequences, the state prioritizes recent inputs, "forgetting" earlier ones due to repeated non-linear squashing.
- There's no mechanism to explicitly retain or prioritize distant but relevant information.
- Vanishing gradients exacerbate this by diminishing the influence of early x_t on later h_T .
- **Example Task:** Next-word prediction in the sentence: "The mouse was getting chased by the cat through the big house. After a long chase, the cat finally managed to catch the" → expected: "mouse". An RNN might predict "cat" instead, having forgotten the early subject "mouse" amid intervening details.

- **Computational Inefficiency:**

The sequential dependency (h_t requires h_{t-1}) prevents parallelization across time steps during training or inference. This contrasts with feedforward networks, where all computations can run in parallel, leading to slower training on GPUs for long sequences.

Key Challenge

*These challenges motivated the development of advanced architectures like gated RNNs, such as **LSTMs**. Prerequisite: Review **Linear Algebra** for gradient concepts.*

Advanced Architectures: Gated RNNs

Gated RNNs

Gated RNNs build on vanilla RNNs by introducing learnable "gates"—sigmoid-activated mechanisms that dynamically control the flow of information. This addresses vanishing gradients and long-term dependencies by allowing the network to selectively remember or forget parts of the state, rather than always overwriting it uniformly.

Common gated variants include:

- **LSTM (Long Short-Term Memory):** Uses multiple gates for precise memory control, excelling in capturing long dependencies.
- **GRU (Gated Recurrent Unit):** A streamlined version of LSTM with fewer gates (update and reset), offering similar performance with less computational overhead.

Key Components of Gated RNNs

Gates operate on values between 0 and 1 (via sigmoid σ), acting as multipliers to retain (1) or discard (0) information:

- **Input Gate:** Regulates how much of the new input x_t should update the memory.
- **Forget Gate:** Determines which elements from the previous memory to discard.
- **Output Gate:** Controls how much of the updated memory contributes to the output y_t and next state h_t .
- **Update Gate (GRU-specific):** Merges input and forget functions for efficiency.

These gates are computed as linear combinations of the current input and previous state, followed by sigmoid activation—e.g., forget gate $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$.

Long Short-Term Memory (LSTM)

An LSTM extends the RNN by introducing a separate **cell state** c_t for long-term memory, which flows linearly with minimal alteration, bypassing non-linearities that cause vanishing gradients. The **hidden state** h_t handles short-term interactions and serves as the output carrier.

Inputs: x_t concatenated with h_{t-1} .

Core Mechanism:

- **Cell State** c_t : Evolves as $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, where $\odot$ is element-wise multiplication, f_t is the forget gate, i_t the input gate, and $\tilde{c}_t$ a candidate update (tanh-activated). This additively updates memory, preserving gradients.
- **Hidden State** h_t : $h_t = o_t \odot \tanh(c_t)$, where o_t is the output gate, blending long-term memory with current context.

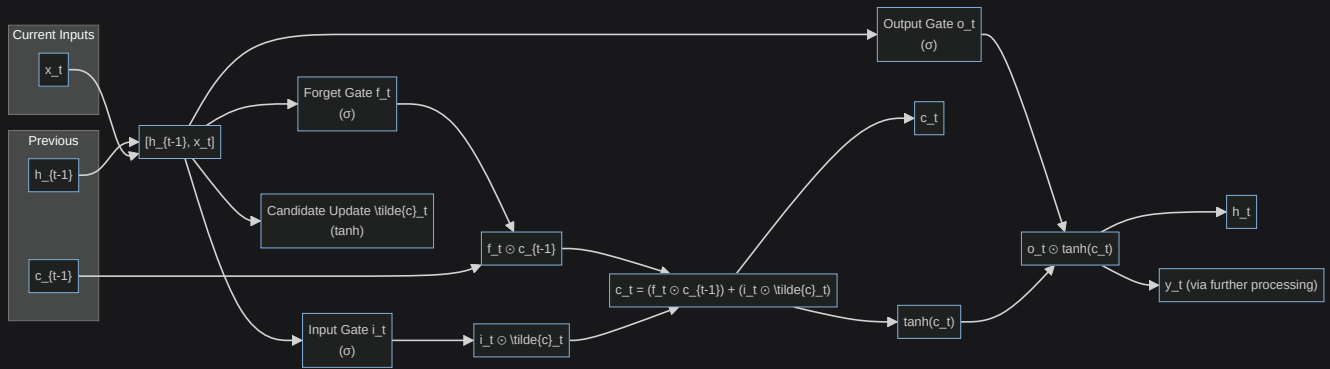
LSTM Gates Computation

(all using linear layers on $[h_{t-1}, x_t]$ + sigmoid):

- **Forget Gate** f_t : $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$; multiplies c_{t-1} (0 = forget all, 1 = retain all).
- **Input Gate** i_t : $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$; scales how much new info $\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ adds to c_t .
- **Output Gate** o_t : $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$; filters c_t into h_t .

In diagrams, each gate is often abstracted as a linear layer + non-linearity (e.g., orange blocks for σ or $\tanh$, with learnable parameters).

For a visual overview of LSTM information flow:



Numerical Example for LSTM

Suppose $c_{t-1} = [1, 0]$, $f_t = [0.2, 0.8]$, $\tilde{c}_t = [0.5, -0.3]$, $i_t = [0.9, 0.1]$. Then $c_t = [0.2 \cdot 1 + 0.9 \cdot 0.5, 0.8 \cdot 0 + 0.1 \cdot (-0.3)] = [0.65, -0.03]$. This selectively retains the first element while minimally updating the second.

Limitations of Gated RNNs

Despite their improvements, gated RNNs like LSTMs and GRUs are not without flaws:

- They offer only marginal gains on very long sequences, as the hidden state remains a fixed-size bottleneck.
- The model processes inputs sequentially, without access to the full future context.
- Gradients can still vanish or explode during BPTT, since the unrolled structure persists.
- Increased complexity: LSTMs add parameters for gates (roughly 4x more than vanilla RNNs), raising computational and overfitting risks.
- Parallelization remains impossible due to sequential dependencies.

Ongoing Limitations

These limitations become evident in sequence-to-sequence (seq2seq) tasks, where input and output lengths vary and global context is crucial. Link to [Deep Learning](#) for advanced seq2seq variants.

Sequence-to-Sequence (Seq2Seq) Tasks

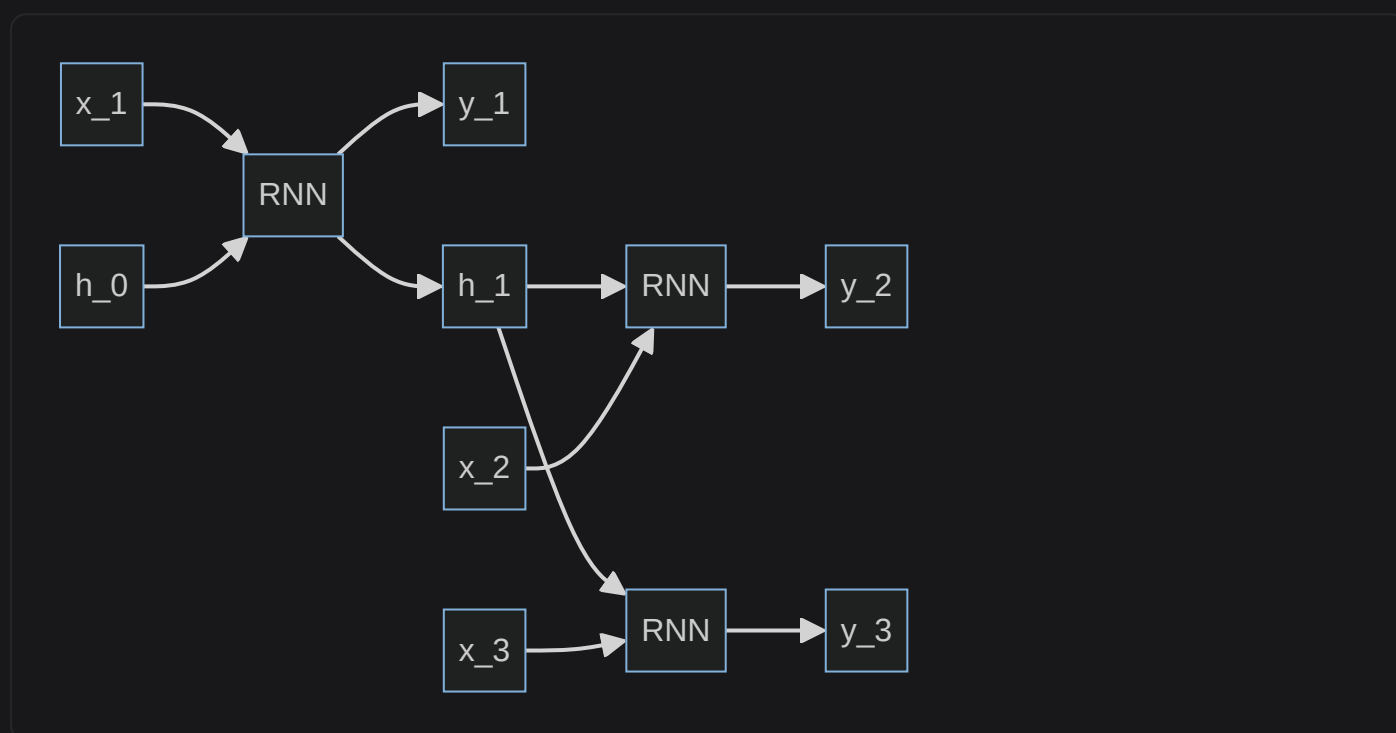
Sequence-to-Sequence (Seq2Seq) Tasks

Seq2seq tasks involve mapping an input sequence to an output sequence, common in applications like machine translation, summarization, and dialogue generation. Early approaches assumed one-to-one mappings (same length inputs and outputs), but real-world tasks often involve length mismatches.

In a basic one-to-one setup:

- Each RNN step processes one input element to produce one output element.
- **Example:** Character-level case correction, where each lowercase input maps to its cased counterpart.

This is visualized as a parallel chain:



Limitations of One-to-One Mapping

While simple, one-to-one mappings falter in complex seq2seq scenarios for two main reasons:

1. Variable Length Mismatches:

Inputs and outputs often differ in length, breaking the strict alignment.

- **Example:** English-to-Italian translation: "it is indeed sunny today" (5 words) → "effettivamente oggi è soleggiato" (4 words). A one-to-one RNN processes word-by-word without adjusting for structural differences, leading to poor alignment.

2. Lack of Full Context:

The decoder generates outputs sequentially without seeing the entire input upfront, missing global information.

- **Example:** In translation, deciding on "effettivamente" (meaning "indeed") requires understanding the full English sentence's nuance, which early steps can't access.

To illustrate with case correction, revisit the input: ""indeed i am sherlock holmes ,""

Target: ""Indeed I am Sherlock Holmes,""

Key casing decisions:

- 1. Capitalize "I" in "Indeed" (after quote).
- 2. Capitalize standalone "I" (pronoun).
- 3. Capitalize "S" in "Sherlock" (proper noun start).
- 4. Capitalize "H" in "Holmes" (proper noun).

In one-to-one:

- Decisions 1 and 4 are feasible with local context.
- But 2 and 3 are hard: The model sees "i" without knowing it's alone, or "s" without "herlock" following.

One-to-One Training Results

Training results (one-to-one RNN, similar setup as demo):

Epoch	Loss	Predicted Output
0001	5.3212	XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
0002	3.7157	XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
0003	2.7964	XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
0004	2.7981	XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
0030	0.2264	" i ndeed i am s herlock h olmes ,"
0050	0.0802	" i ndeed i am s herlock h olmes ,"
0160	0.0301	"Indeed i am s herlock Holmes,"
0530	0.0224	"Indeed i am s herlock Holmes,"
1070	0.0232	"Indeed i am s herlock Holmes,"

The model handles "Indeed" and "Holmes" via patterns but fails on "I" and "Sherlock" due to lack of foresight.

Encoder-Decoder Architecture

Encoder-Decoder Architecture

The encoder-decoder framework overcomes these issues by decoupling encoding (summarizing input) from decoding (generating output):

1. The encoder processes the full input to produce a fixed-size context vector.
2. The decoder initializes from this vector and generates the output sequence autoregressively.
3. Generation stops at an end-of-sequence (EOS) token.

Components:

- **Encoder:** A (gated) RNN that reads the entire input sequence, updating its state until a final context vector z (often the last h_T).
- **Decoder:** Another RNN starting from z + a beginning-of-sequence (BOS) token, predicting tokens one-by-one using previous outputs as inputs.

Encoder-Decoder in Translation

(English to Italian translation):

Encoder: Processes "it" → "is" → "indeed" → "sunny" → "today" → final state z .

Decoder: z + <BOS> → "effettivamente" → "oggi" → "è" → "soleggiato" → <EOS>.

- The first decoder step uses <BOS>; subsequent steps use predicted (or teacher-forced true) tokens.
- **Teacher Forcing in Training:** Feed ground-truth previous outputs to speed convergence and reduce error accumulation.

For case correction with encoder-decoder (same input/target):

Epoch/Loss	Predicted Output
5.4591	wwXXXXXXXXXXXXXXXXXXXXXXXXXXXX
4.0161	aXXXXXXXXXXXXXXXXXXXXXXXXXXXX
2.8268	XXXXXXXXXXXXXXXXXXXXXXXXXXXX
2.7671	XXXXXXXXXXXXXXXXXXXX

Epoch/Loss	Predicted Output
0.2315	" i ndeed i am s herlock h olmes,"
0.0705	" i nd

References

- [Neural Networks](#)
- [Machine Learning](#)
- [Deep Learning](#)
- [Linear Algebra](#)